

Multi-model cascade router

Individual printable topic report.

Generated 2026-06-17

Why I built it

Agent systems fail when all tasks are pushed through one model. The router treats models as resources with strengths, costs, quotas, cooldowns, and failure modes.

Architecture and evidence

``ai_router.py``, ``cascade.py``, and ``litellm_config.yaml`` define model registries, routing strategies, tiering, quota tracking, fallback, race/cancel behavior, and budget-aware escalation.

Claim boundary: Cost-reduction claims should be treated as design goals unless fresh logs verify exact percentages.

- Task strategy selects speed, quality, free, code, creative, bulk, or local routing.
- Local models are tried before free APIs and paid tiers where appropriate.
- Quota/cooldown tracking blocks bad or unavailable routes.
- Quality gates decide whether to escalate.
- Free API models can be raced in parallel and slower calls cancelled after a sufficient answer.
- Outcomes are logged for later learning.

What I can show with this

- Model selection was treated as an engineering system rather than a preference.
- The design accounts for cost, quotas, rate limits, cooldowns, historical quality, and fallback.
- The router connects local/offline models with API/CLI providers.
- Parallel racing and cancellation were considered for free-model tiers.
- A strong agent stack should degrade gracefully when models fail.
- Quality gates should decide escalation, not model status alone.
- Cost and privacy are part of capability routing.
- Outcome logging is the seed of empirical model governance.
- Run a fresh benchmark across representative task classes.
- Publish aggregate cost/latency/quality results without provider secrets.
- Add policy labels for private, public, cheap, fast, and high-stakes tasks.
- Use the router as the basis for a secure-agent evaluation project.